Table 4: **Reward Modeling ability improves with Self-Training**: We evaluate the LLM-as-a-Judge via various metrics which measure alignment with held-out human preference data. Self-Rewarding Iteration 2 (Model M_2), which is trained using the self-reward model derived from its previous iteration M_1 outperforms Iteration 1 (M_1), while M_1 itself outperforms a standard SFT baseline model trained on only Instruction Fine-Tuning (IFT) data. Iteration 3 (Model M_3) gives further improvements over Iteration 2.

		Self-Rewarding Models		
Model	SFT Baseline	Iter 1 (M_1)	Iter 2 (M_2)	Iter 3 (M_3)
Training data	IFT	IFT+EFT	IFT+EFT +AIFT(M_1)	IFT+EFT+AIFT(M_1) +AIFT(M_2)
Pairwise acc. (↑)	65.1%	78.7%	80.4%	81.7%
5-best % (↑)	39.6%	41.5%	44.3%	43.2%
Exact Match % (↑)	10.1%	13.1%	14.3%	14.3%
Spearman corr. (↑)	0.253	0.279	0.331	0.349
Kendall τ corr. (↑)	0.233	0.253	0.315	0.324

again see improvements across the iterations of training from M_1 to M_3, from 6.78 (out of 10) up to 7.25, with larger relative gains in the humanities, STEM, roleplay, writing and extraction categories, and smaller gains in the math, code and reasoning categories. We expect that the latter is due to the seed prompts we use from Open Assistant tending to underemphasize the reasoning-based tasks. We note also that these improvements are in spite of our method using and constructing prompts that only involve a single turn, given the MT-Bench benchmark itself is a multi-turn evaluation.

Self-rewarding models did not lose ability on NLP Benchmarks As shown in Table 3, the performance of most NLP benchmark tasks evaluated are roughly similar to the baselines, with further detailed results on more datasets given in Appendix Table 9 that follow the same pattern. We hypothesize that given that our training data (seed data and synthetically generated data) are based on the Open Assistant prompts which may not be especially relevant to skills needed in the Table 3 tasks, it is expected that the task performance stays roughly similar, or may even drop. For example, in InstructGPT training [Ouyang et al., 2022] they found that "during RLHF fine-tuning, we observe performance regressions compared to GPT-3 on certain public NLP datasets" which they refer to as an "alignment tax." A clear future direction is to extend the self-rewarding paradigm to these types of tasks, by relying not only on seed prompts from Open Assistant, but also on seed prompts found in a larger variety of datasets.

3.2.2 Reward Modeling Ability

Reward modeling evaluation results are provided in Table 4.

EFT augmentation improves over SFT baseline Firstly, we find that adding Evaluation Fine-Tuning (EFT) data into training, which gives examples to the model of how to act as an LLM-as-a-Judge, naturally improves its performance compared to training with Instruction Fine-Tuning (IFT) data alone. IFT data covers a wide range of general instruction tasks, and so does endow the SFT Baseline with the ability to evaluate responses; however, EFT data gives more examples of this specific task. We find improvements across all five metrics measured when using IFT+EFT vs. IFT alone, e.g., the pairwise accuracy agreement with humans increases from 65.1% to 78.7%.

Reward Modeling ability improves with Self-Training We find that performing a round of self-reward training *improves the ability of the model at providing self-rewards for the next iteration*, in addition to its improved instruction following ability. Model M_2 (Iteration 2) is trained using the reward model from M_1 (Iteration 1), but provides improved performance on all five metrics compared to M_1. For example, pairwise accuracy improves

from 78.7% to 80.4%. Iteration 3 (M_3) improves several of these metrics further compared to M_2, for example pairwise accuracy increases from 80.4% to 81.7%. This performance gain is achieved despite there being no additional EFT data provided, and the examples created during the *Self-Instruction creation* loop do not tend to look like LLM-as-a-Judge training examples. We hypothesize that because the model is becoming better at general instruction following, it nevertheless also improves at the LLM-as-a-Judge task.

Importance of the LLM-as-a-Judge Prompt In these experiments we used the LLM-as-a-Judge prompt format shown in Figure 2. In preliminary experiments we also tried various other prompts to decide the most effective one to use. For example, we tried the prompt proposed in Li et al. [2024] which also proposes a 5-point scale, but describes the options as multiple choice in a range of quality buckets, see Appendix Figure 7. In contrast, our prompt describes the points as additive, covering various aspects of quality. We find a large difference between these two prompts when using the SFT Baseline, e.g. 65.1% pairwise accuracy for ours, and only 26.6% pairwise accuracy for theirs. See Appendix A.2 for further details.

4 Related Work

Automatically improving or self-correcting large language models is becoming a major focus of research. A recent survey from Pan et al. [2023] attempts to summarize the topic. However, this is a rapidly moving area, and there are already promising new works not covered there.

Reinforcement Learning from Human Feedback (RLHF) Preference learning approaches such as in Ziegler et al. [2019], Stiennon et al. [2020], Ouyang et al. [2022], Bai et al. [2022a] train a fixed reward model from human preference data, and then use the reward model to train via reinforcement learning (RL), e.g. via Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Thus, the reward signal in a certain sense already comes from a model even in these works, but distilled from human data. Nevertheless, this is commonly referred to as RL from Human Feedback (RLHF). Methods such as Direct Preference Optimization (DPO) [Rafailov et al., 2023] avoid training the reward model entirely, and instead directly train the LLM using human preferences. Several other such competing methods exist as well [Zhao et al., 2023, Zheng et al., 2023a, Yuan et al., 2023], including Pairwise Cringe Optimization (PCO) [Xu et al., 2023]. PCO uses an iterative training approach similar to the one in our work, except with a fixed reward model, and that work also showed that Iterative DPO improves over DPO using the same scheme. We note that other works have developed iterative preference training schemes as well, e.g. Adolphs et al. [2023], Gulcehre et al. [2023], Xiong et al. [2023].

Reinforcement Learning from AI Feedback (RLAIF) Constitutional AI [Bai et al., 2022b] uses an LLM to give feedback and refine responses, and uses this data to train a reward model. This fixed, separate reward model is then used to train the language model via RL, called "RL from AI Feedback" (RLAIF). Lee et al. [2023] compare RLAIF and RLHF procedures and find the methods they compare perform roughly equally. They use an "off-the-shelf" LLM to perform LLM-as-a-Judge prompting to build a training set to train a fixed reward model, which is then used for RL training. They also experiment with using the fixed but separate LLM-as-a-Judge model directly, which the authors report is computationally expensive due to using it within PPO training (rather than the offline step in the iterative approach we use in our work, which is relatively computationally cheap). Finally, SPIN [Chen et al., 2024b] recently showed they can avoid reward models entirely in an Iterative DPO-like framework by using human labels as the winning response in a pair, and the last iteration's generations as the losing response in the pair. The authors note this has the limitation that once the model generations reach human performance, they are bottlenecked. Further, each input prompt is required to have a human annotated response, in contrast to our work.

Improving LLMs via data augmentation (and curation) Several methods have improved LLMs by (self-)creating training data to augment fine-tuning. Self-Instruct [Wang

et al., 2023] is a method for self-instruction creation of prompts and responses, which can be used to improve a base LLM. We make use of a similar technique in our work, and then use our self-reward model to score them. Several approaches have also created training data by distilling from powerful LLMs, and shown a weaker LLM can then perform well. For example, Alpaca [Taori et al., 2023] fine-tuned a Llama 7B model with text-davinci-003 instructions created in the style of self-instruct. Alpagasus [Chen et al., 2024a] employed a strong LLM-as-a-Judge (ChatGPT) to curate the Alpaca dataset and filter to a smaller set, obtaining improved results. Instruction Backtranslation [Li et al., 2024] similarly augments and curates training data, but augmenting via backtranslating from web documents to predict prompts. The curation is done by the LLM(-as-a-Judge) itself, so can be seen as an instance of a self-rewarding model, but in a specialized setting. Reinforced Self-Training (ReST) [Gulcehre et al., 2023] uses a fixed, external reward to curate new high-quality examples to iteratively add to the training set, improving performance. In our experiments, we found that adding only positive examples in a related manner did not help, whereas preference pairs did help (see Appendix Section A.4 for details).

LLM-as-a-Judge Using LLM-as-a-Judge prompting to evaluate language models has become a standard approach [Dubois et al., 2023, Li et al., 2023, Fernandes et al., 2023, Bai et al., 2023, Saha et al., 2023], and is being used to train reward models or curate data as well, as described above [Lee et al., 2023, Chen et al., 2024a, Li et al., 2024]. While some works such as Kim et al. [2023] create training data to train an LLM to perform well as a judge, to our knowledge it is not common to combine this training with general instruction following skills as in our work.

5 Conclusion

We have introduced Self-Rewarding Language Models, models capable of self-alignment via judging and training on their own generations. The method learns in an iterative manner, where in each iteration the model creates its own preference-based instruction training data. This is done by assigning rewards to its own generations via LLM-as-a-Judge prompting, and using Iterative DPO to train on the preferences. We showed that this training both improves the instruction following capability of the model, as well as its reward-modeling ability across the iterations. While there are many avenues left unexplored, we believe this is exciting because this means the model is better able to assign rewards in future iterations for improving instruction following – a kind of virtuous circle. While this improvement likely saturates in realistic scenarios, it still allows for the possibility of continual improvement beyond the human preferences that are typically used to build reward models and instruction following models today.

6 Limitations

While we have obtained promising experimental results, we currently consider them preliminary because there are many avenues yet to explore, among them the topics of further evaluation, including safety evaluation, and understanding the limits of iterative training.

We showed that the iterations of training improve both instruction following and reward modeling ability, but only ran three iterations in a single setting. A clear line of further research is to understand the "scaling laws" of this effect both for more iterations, and with different language models with more or less capabilities in different settings.

We observed an increase in length in model generations, and there is a known correlation between length and estimated quality, which is a topic that should be understood more deeply in general, and in our results in particular as well. It would also be good to understand if so-called "reward-hacking" can happen within our framework, and in what circumstances. As we are using both a language model as the training reward, and a language model for final evaluation (GPT-4) in some of our benchmarks, even if they are different models, this may require a deeper analysis than we have provided. While the human evaluation we conducted did provide validation of the automatic results, further study could bring more insights.